Promo Finder

Kuku AI Generalist Assignment, Problem 1 · Nithish Ravikkumar · Enrollment no. 23123028

Live demo: promo-finder-nrk.streamlit.app · Code: github.com/shadowPunch/promo-finder

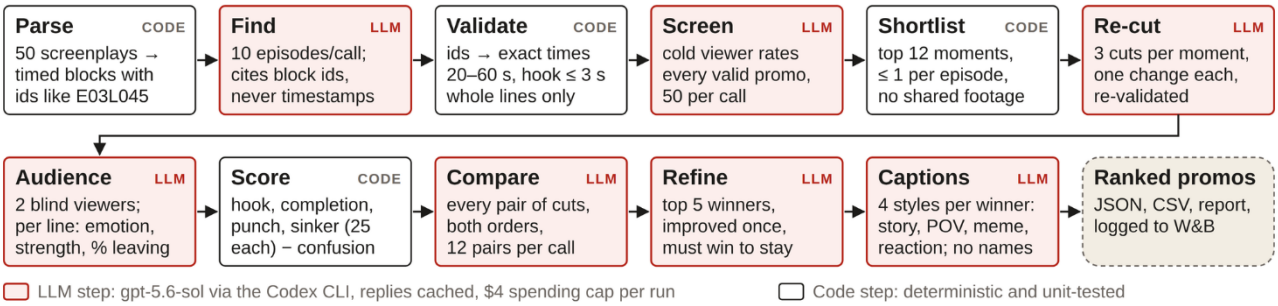
Problem Statement

To make promos for shows, an editor watches every episode of every show to find a handful of 20–60 second clips. Kuku releases new shows every week. The work is repetitive, doesn't scale, but is necessary as it feeds straight into user acquisition. Its output is also easy to check: a timestamp is right or wrong, and a hook lands in the first three seconds or it doesn't. So this can be automated and be tested for correctness easily. What I have built is a pipeline that takes in script transcripts and outputs recommended promo script cuts.

Assumptions

- I only have the script, so an editor still checks each promo against the video for visual hooks.
- Promos are cut straight from episodes (no narration, music or text): 20–60 s, a hook within 3 s, ending on the cliffhanger.
- The main ad audience is women 18–30 who like romance and revenge drama, weighted 65% in the score, plus casual newcomers at 35%. This is based on market research done on global audience, not Indian market, where the ratio is probably inverted in favour of men. However, for the purposes of this assignment, considering that the script is in English and Indian audience mostly consumes Hindi content, my pipeline assumes global English audience. Also, modifying the viewer preferences for a different audience can be done easily in my pipeline.
- The score only predicts hook rate and completion, which might not give full picture which can only be obtained by market testing.

Pipeline



1. **Rule based validation.** The parser turns each screenplay into timed blocks with ids. Models cite block ids in JSON, never times; code maps them to exact times and enforces the rules (20–60 s, hook within 3 s, ≤ 2 s after the cliffhanger, 1–3 non-overlapping segments, whole lines only).
2. **Audience and score.** Viewer A is the core fan (weight 0.65), viewer B a newcomer (0.35). Per line, each reports an emotion, its strength (0–3) and the share scrolling away, then imagines 2–4 next scenes with likelihoods and rates need-to-know (1–5). The score gives 25 points each to hook (share watching at 3 s), completion, punch (peak emotion strength) and cliffhanger pull (need-to-know × spread of imagined scenes), minus up to 15 for confusion.
3. **One-on-one comparisons.** Every pair of a moment's cuts is judged by both viewers in both orders, in separate calls, so first-position bias cancels out; the cut with the highest weighted win share wins.
4. **Model calls.** Swappable model backends (AgentRouter call through Codex CLI or any OpenAI-compatible API, via .env) and cached replies for efficient usage.

The model handles taste: moments, re-cuts and viewer reactions. Python scripting is used for rule based verification and validation to prevent hallucination, from timestamps to scoring, and an editor approves the final list.

Four business frameworks shaped the design. The score follows **AIDA** and works as a **North Star metric** tied to platform numbers. The pipeline is a **stage-gate**, with a cheap screen before costly tests. Each re-cut changes one thing (**test-and-learn**), so every win can be explained.

Results on the 50 episodes

Of 93 proposed promos, 92 passed the rules. I tested 12 moments in 50 cuts and ran 140 one-on-one comparisons. A re-cut beat the original in 7 of the 12 moments, mostly through a stronger opening (it won 5 of 11 tries); new endings, lengths and structures never won. The three I'd post first:

Clip	Hook and cliffhanger	Score	Won
E41 01:21.0–01:36.1 + E42 00:11.2–00:28.8 (33 s, refined)	An explosion tears open the cathedral. Who is the stranger the royal guards obey?	78.5	100%
E36 02:14.5–02:43.0 + 02:59.5–03:16.8 (46 s, new opening)	His mother tells him his wife is to be executed. What will he do when he reaches the wedding?	72.2	91%
E12 00:57.3–01:24.7 (27 s, new opening)	A tormentor orders her to retrieve something with her bare hands. Can she survive and save her husband?	72.0	73%

Evaluation

The rules hold by construction: every cut lands on a real line boundary, and 112 unit tests with fake models check the timing and rule logic.

The comparisons came out of a failure. I first picked each moment's best cut by its score out of 100, but when the same cuts were scored twice, the two passes agreed on which of two cuts was better only 61% of the time. Asking “which of these two would you keep watching?” turned out to be far steadier:

Check	Score out of 100	One-on-one
Same choice when a pair is judged twice (chance is 50%)	61%	86%
Same best cut from two independent rounds	1 of 6 moments	8 of 12 moments
Cut shown first wins (no bias is 50%)	–	49%

Limitations

1. **Limited labels.** Every promo has to fit one of four types: explosive, funny, puzzling or shocking. Plenty of good moments don't, and the finder never labelled a single promo funny. Episodes 6, 10 and 17 got no promo at all. The label set should be expanded.
2. **Heavy dependence on the opening.** Only new openings beat the original cut. The new endings, lengths and structures never won, so in practice those options went unused.
3. **No human feedback, no video.** No real viewer has judged these promos, and the simulated viewer sees speaker names a real viewer never would. AI viewers agree only weakly with people in published tests. Editor feedback and visuals from the footage would make much better cut decisions.

Models used

I used Claude Code for research and writing code. Also, every model call used gpt-5.6-sol through AgentRouter, called via the Codex CLI, and it all ran on free credit: about $16.40 over the whole project, with no paid spend.

Much of the cost is overhead: the Codex CLI adds about 17K tokens of instructions to every call. This was done because AgentRouter can only be used via a harness with interchangeable base model. If this is to be done at scale, then using paid API calls, we can avoid the overhead. At Kuku's volume, promos for each show can be used as testing material, as we analyze marketing metrics for each promo. We can get a ranked list that goes to an editor's queue. Their decisions become test data, and real hook rates from posted promos can tune the score.

Also, with more budget, video models that watch the footage could replace the script-only viewer.